HARP-PAF | Classroom Run Guide

Ubuntu terminal only - no VS Code required - commands in the exact order to run

Goal: Start the local HARP-PAF backend and frontend from two Ubuntu terminals, open the website in a browser, and show the complete demonstration. These steps use your existing files in /home/jimk7521/harp-paf.

Important: Running the app does not download the code again. Do not run git clone, npm install, pip install, or code . for a normal presentation. Your existing Python virtual environment and Node dependencies should already be installed.

1. Before you enter the classroom

Use the same Windows laptop that has Ubuntu (WSL) and the HARP-PAF folder installed. Open the Windows Start menu, search Ubuntu, and open it. Internet is not required for the local application once all dependencies are installed.

In the Ubuntu window, confirm your project exists:

```
cd ~/harp-paf
pwd
ls backend frontend examples
```

Expected location: /home/jimk7521/harp-paf. The last command should show the three folders. If it does, continue.

2. Ubuntu Terminal 1 - start the backend

In the first Ubuntu terminal, enter these commands one by one:

```
cd ~/harp-paf
source venv/bin/activate
uvicorn backend.app:app --host 127.0.0.1 --port 8000
```

The prompt may show (venv) after activation. Leave this terminal open and running. It is the Python/FastAPI backend. Do not enter frontend commands in this terminal.

Optional backend check in a browser: http://127.0.0.1:8000/health.

3. Ubuntu Terminal 2 - start the website

Open a second Ubuntu window from the Windows Start menu. Do not close Terminal 1. In Terminal 2 run:

```
cd ~/harp-paf/frontend
npm run dev
```

Wait until Vite prints a local URL, typically http://localhost:5173/. Leave this terminal open and running. This is the React frontend.

Tip: If Vite displays a different port (for example 5174), use the URL it actually prints rather than assuming 5173.

4. Open the demonstration

Open Chrome or Brave on Windows and visit http://localhost:5173/ (or the Vite URL). Keep both Ubuntu terminals running behind the browser for the whole demo.

Presentation flow | what to click and say

5. Demonstrate the application in class

Suggested 6-8 minute walkthrough:

1. Home / dashboard: Introduce HARP-PAF as a local prototype for analyzing four C++ workloads and choosing pairs to execute together.

2. Upload: Select the four approved C++ example source files, task1.cpp, task2.cpp, task3.cpp, and task4.cpp. Start processing and wait for the job to complete.

3. Tasks: Show the workload information and the measured task details.

4. Pairs: Show how the tool ranks/selects pairs and displays their heuristic compatibility.

5. Results: Show the measured baseline and selected pair run times and observed speedup. Explain that small timing differences on WSL are demonstration results, not definitive hardware-performance proof.

6. Predictions: Point out the heuristic scores and clearly distinguish them from measured outcomes. Hardware counters or validated prediction models are not implemented.

7. History: Show saved job records so you have a backup demonstration if time runs short.

8. System: Show the local WSL environment, available logical CPUs, and compiler/runtime information.

6. Selecting the four C++ files from Windows

When the browser opens a Windows file-selection dialog, paste this location in the dialog address bar:

```
\\wsl.localhost\Ubuntu\home\jimk7521\harp-paf\examples
```

Choose these source files: task1.cpp, task2.cpp, task3.cpp, task4.cpp. Do not choose similarly named files without the .cpp extension; those are compiled executables.

The folder name Ubuntu assumes that is the name of your installed WSL distribution. If the path does not open, use Windows File Explorer and open the Linux / Ubuntu files area to locate the same examples folder.

7. Short presentation script

“Ma’am, this is HARP-PAF, a local prototype for phase-aware resource pairing. I upload four approved C++ programs, and the backend compiles and analyzes them. The system uses workload features to form candidate pairings, runs a baseline and selected pairing, and reports measured timing results. The website also shows task details, pairing scores, results, history, and environment information. The present version is a working research demonstration on WSL, not a fully calibrated production scheduler.”

8. End the presentation safely

In Terminal 2 (frontend), press Ctrl+C. In Terminal 1 (backend), press Ctrl+C, then optionally type:

```
deactivate
exit
```

In Terminal 2, after stopping Vite, you may type exit to close it. The project files and saved job results remain on disk.

Troubleshooting | classroom quick fixes

The website does not open

Confirm Terminal 2 still shows Vite running. Open the exact local URL printed by Vite. Check that you did not close the terminal or press Ctrl+C.

The website opens, but uploading or results fail

Confirm Terminal 1 still shows Uvicorn/FastAPI running on port 8000. Restart the backend in Terminal 1 if needed:

```
cd ~/harp-paf
source venv/bin/activate
uvicorn backend.app:app --host 127.0.0.1 --port 8000
```

“Address already in use” (port 8000)

A backend may already be running. In a separate Ubuntu terminal, check:

```
curl http://127.0.0.1:8000/health
```

If you receive a healthy response, use that existing backend instead of starting another one. Do not kill processes without first identifying them.

“npm: command not found” or “uvicorn: command not found”

These indicate that the expected tools are not available in the current terminal or environment. For the backend, verify the virtual environment was activated. Check the commands below; if a tool is missing, correct the installation before class rather than attempting a fresh download during the presentation.

```
cd ~/harp-paf
source venv/bin/activate
which python
which uvicorn
node --version
npm --version
```

“Cannot find module” in frontend

Check that you are running npm run dev inside ~/harp-paf/frontend, and that the frontend/node_modules directory exists. Do not run npm install unless you actually need to repair dependencies.

Demo takes too long

Use the History, Results and Pairs pages to explain a previously completed job. A live benchmarking run may take time and its measured timings can vary.

Last-minute checklist

Laptop powered/charged; Ubuntu opens; both terminals can launch; health check works; browser opens the local Vite page; four .cpp files are available; at least one completed job exists in History; know how to stop both terminals with Ctrl+C.

One-screen command cheat sheet

Ubuntu Terminal 1 - backend:

```
cd ~/harp-paf
source venv/bin/activate
uvicorn backend.app:app --host 127.0.0.1 --port 8000
```

Ubuntu Terminal 2 - frontend:

```
cd ~/harp-paf/frontend
npm run dev
```

Windows browser: http://localhost:5173/ | Stop: Ctrl+C in both Ubuntu terminals